Drifting Away From The Teacher: Rewarding Diversity in Answers via Custom GRPO Reward Functions

Göksel OKANDAN
Computer Engineering
Yıldız Technical University
Istanbul, Turkiye
goksel.okandan@std.yildiz.edu.tr

***Abstract*—This study explores the comparison of diversity-focused reward functions in GRPO trainings during mathematical reasoning tasks. A Qwen3.5-4B base model was first distilled using a teacher model (gpt-oss-120b) on a Turkish dataset (ytu-ce-cosmos/gsm8k_tr). Two GRPO models were trained afterwards with the first focused only on accuracy, and the second focused on accuracy and diversity both, encouraging diversity with a custom reward function that used Jaccard Similarity as a proxy to Kullback-Leibler divergence penalty that is inherent in GRPO. Evaluation results indicated a steep drop in mathematical accuracy for both GRPO models (below 2%) compared to the distilled model (44.12%), alongside low reasoning diversity across all models. However, manual analysis revealed that the accuracy collapse and low diversity coefficients were products of evaluation fragility and reward hacking via formatting artifacts instead of reasoning degradation. These findings highlight the fact that while small sized models are capable of solving mathematical questions, testing methods that can compensate for formatting artifacts and cross-lingual differences are highly necessary for future evaluation processes.**

***Index Terms*—machine learning, answer diversity, GRPO training, computational semantics**

I. Introduction

With advancements in technology in the past decade, the reasoning capabilities of large language models have increased exponentially. What once could only provide outputs based on established rules alone is now capable of solving complex problems that would require tremendous effort from humans without it. This is in no small part due to the training methods established via countless hours, trials and errors. Two of these important training methods are supervised fine-tuning (SFT) and group relative policy optimization (GRPO).

Supervised Fine-Tuning (SFT) is the process of using a pre-trained language model and further training the same model on a smaller, task-specific dataset with labeled examples with the goal of adjusting the weights of the model to show better performance on specific task defined without losing aptitude at other disciplines [1]. Meanwhile, Group Relative Policy Optimization (GRPO) is a reinforcement learning technique that has gained widespread appraisal after the emergence of DeepSeek-R1. GRPO was first introduced in DeepSeekMath, a model that was specifically fine-tuned for solving mathematical equations and problems via advanced reasoning, while being originally designed for improved efficiency in fine-tuning [2]. GRPO capable of achieving this efficiency in fine-tuning tweaking the existing policy via introducing a reference policy and using a statistical distance technique known as Kullback-Leibler divergence, which helps the model maintain stability while enabling the model to explore strategies [3].

In this project, the impact of introducing rewards for providing divergent answers to the baseline answers provided by a teacher model in GRPO training compared to a standard, mathematical accuracy focused GRPO training has been investigated. Using a distilled **Qwen3.5-4B** model as a baseline student model [4], two separate GRPO training sessions have been performed on this model, with one focusing only on the mathematical accuracy of the provided answers, and the other one focusing on the diversity of the provided answers along with the mathematical accuracy of these models, using Jaccard Similarity method as a proxy to the Kullback-Leibler modifications mentioned in the project document. The dataset used in this project was the **ytu-ce-cosmos/gsm8ktr** dataset [5], which was developed by the Yıldız Technical University's COSMOS NLP Research Group. Additionally, the teacher model that was used to establish baseline reasoning paths for the student models was OpenAI's **gpt-oss-120b** [6] model, despite contradicting instructions in the project document.

The following sections outline the methodology of this comparative training pipeline, present the evaluation metrics regarding accuracy and reasoning diversity, discuss the potential reasons for the results provided (especially the reasons for low evaluation scores in accuracy and diversity), and provide potential future tasks for the project.

II. Methodology

This section highlights the development steps and the methods performed during the development of the project, which was developed with the intent to compare and highlight the effects of specfically encouraging a model to give diverse answers during GRPO training. The development flowchart followed can be found below (Fig. 1).

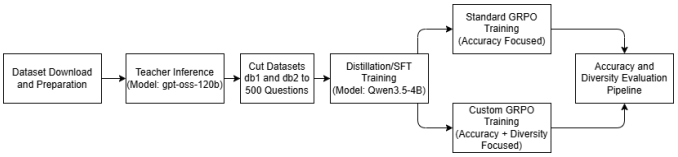

Fig. 1. The development flowchart of the project.

A. Preparation of the Project Environment and the Datasets

The training split of the **ytu-ce-cosmos/gsm8ktr** dataset was downloaded from Huggingface, with the dataset being further split into the three datasets (**db1**, **db2**, and **db3**), with each of the datasets having 500 questions at the start.

Despite the project document stating the teacher model as **Qwen3.6-27B** [7], the teacher model was switched to OpenAI's **gpt-oss-120b** model due to several issues encountered with Qwen-3.6-27B during the inference procedure (it wasn't able to format the answers as stated in the prompt, or was unable to provide correct answers). This model was able to answer 491 of the 500 questions in the appropriate format during the first inference test that used the questions in db1. After the results of the first few inference tests with both db1 and db2 (both datasets were used during the teacher inference procedure in order to get the teacher model's answers for both datasets separately in order), and the impossibility of the model answering all questions with the provided answer format, the number of questions in the datasets db1 and db2 were increased to 550 questions, with the first 500 questions that were answered in the proper format being separated as the final db1 and db2 datasets. After the teacher model's inference process was run for both of the datasets db1 and db2, the resulting .jsonl files were saved to be used during the distillation and the GRPO training processes.

B. The Base Model and The Distillation Process

The base model that was used for this project was Qwen's **Qwen3.5-4B** model, as it was stated in section I and the project document.

The distillation process consisted of just performing supervised fine-tuning (SFT) on the base model with the questions and answers that the teacher model provided via the inference procedure that was mentioned in section II-A. After loading the .jsonl file that contained the questions and answers of db1, the SFT training parameters were determined (notable settings being a learning rate of 1e-4, 5 epochs, and a max length of 1536 tokens among other settings) and the supervised fine-tuning session began with additional LoRA (Low-Rank Adaptation)/PEFT (Performance Efficient Fine-Tuning) configurations. A chat template involving the system prompt as "system", the question as "user", and the teacher model's answer as "assistant" was also used for this SFT procedure. The trained/distilled model was then saved and would be reused during the rest of the development process.

C. GRPO Training Process and the Reward Functions

1) GRPO Environment Setup: As stated in the project document, the distilled model was to be used as the basis for two kinds of GRPO trainings, one which focused only on the answers being correct with the other one being focused on both the accuracy and the diversity of the of the answers (specifically on the answers being divergent from the teacher model's). In order to facilitate both training sessions in a reasonable amount of time, especially after setbacks encountered with the Colab service being unavailable for several days, faster fine-tuning methods and libraries, such as Unsloth, were implemented prior to the start of both of the GRPO training sessions.

2) Standard GRPO Training Procedure: After Unsloth was setup, the training process for the standard GRPO, which was only focused on tuning the model for further accuracy, was started with the appropriate parameters (most notably, a learning rate of 5e-6, 1 epoch only, 0.7 temperature rating, max prompt length of 512 and max completion length of 1280, with the tokenizer having a 2048 token limit among other settings). Additionally two reward functions were setup, *"format_reward_func"* (reward of 1.0 if format appropriate (`<answer>`answer number without units or words`</answer>`), 0.0 if not) and *"accuracy_reward_func"* (reward of 2.0 if accurate, 1.0 if calculation is there, 0.0 if there is no calculation or the answer). These rewards were additive, so a fully formatted and accurate answer would get 3.0 points. In addition to these functions, a function named *"brevity_reward_func"* was introduced, with its role being to punish answers that had more than 1000 characters to discourage the model from giving extremely long answers. Additionally the "ground_truth" parameter of the *"accuracy_reward_func"* function was changed to the parameter "teacher_answer" to prevent the accuracy reward not giving any rewards, which proved itself to be an issue during development.

3) Custom GRPO Training Procedure: After the standard GRPO process was finished, the code from the standard GRPO training session was then ported without changes in order to provide the same conditions for each of the training processes, with the exception of the *"teacher_divergence_reward_func"*, which was used to reward the model for giving answers that diverged from the teacher model's answer, with reward values of 1.5 for divergence above 0.85 (85%), -0.5 for divergence less than 0.30 (30%), and 0.5 for a divergence score between these two values.

The diversity calculation's primary method was the Jaccard Similarity function. This decision was caused by the Kullback-Leibler divergence penalty calculation ($\beta D_{KL}(\pi_\theta||\pi_{ref})$, with π_θ representing the current policy of the model and π_{ref}) representing the reference model's policy) already existing inside the the loss function that is used for all the GRPO training sessions (with the "beta" parameter, which is the anchor value for Kullback-Leibler divergence penalty, adjusted to 0.04), and making changes inside this function could provide extremely

negative effects to the training session, such as causing a policy collapse. In order to avoid these risks that could jeopardize the custom training process, the Jaccard Similarity method was actually used as a proxy for the Kullback-Leibler divergence penalty in order to calculate the lexical diversity between the answers of the student and the teacher model.

In addition to using Jaccard Similarity as a proxy, the compared parts of the student and the teacher model's answers were changed from the "ground_truth" section to the "teacher_answer" in order to compare English chain-of-thoughts of both the student and the teacher model. This was caused by the teacher model (gpt-oss-120b) reasoning in Turkish when solving the problems provided to it, rather than in English despite being prompted to reason in English.

With both of these changes made, the custom GRPO training session ran without encountering any further issues, with the resulting model saved and used in the next step of the project.

D. The Model Evaluation Process

After the GRPO trainings were finished with both models resulting out of them saved, the only step of the project that needed completion was the evaluation process. In order to complete this process, a pipeline that loaded the distilled model and the two GRPO trained models along with the dataset that included the test questions (db3 with the 500 questions inside), generated answers to questions provided in db3, evaluated the answers that the models generated in terms of accuracy and diversity and printed the results for each model, and generating the plots for each model's performance and merging them together as the final step.

The models were pulled from the developer's personal Google Drive in order to provide permanent storage due to Colab having temporary storage for each different runtime, requiring the developer's Drive to be mounted onto Colab. Additionally, in order to save time while using more of the available resources of the runtime's GPU (the 80GB version of the Nvidia A100 GPU), questions were provided in batches of 32 for each of the models to solve. After the answer generation process of each model was over, the answers that each of the models gave were subjected to accuracy evaluation via comparing the values that were pulled from the `<answer>` and `</answer>` tags (for models) with the regex extracted last number of the relevant question's answer (as it was discovered to be a regular occurrence with the dataset used). In addition to the accuracy evaluation via string comparison, the diversity of the answers provided by the models was also under evaluation, with the process being carried out with the calculation of cosine similarity via an embedding model (**Qwen3-Embedding-0.6B** [8] in this project), then the cosine similarity value being subtracted from 1.0 (the maximum similarity value).

After the end of the evaluation, the results for each model was printed along with an example of failure for the custom trained GRPO model (the reason will be explained in section IV of the report) and the performance plots of each of the models (Fig. 2) were created using the **matplotlib** library.

III. Results

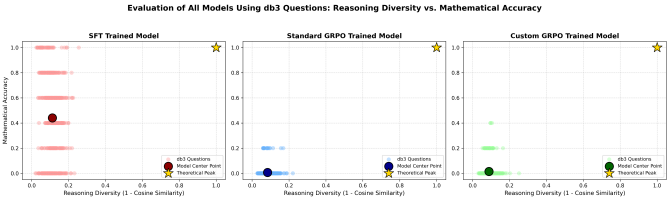

Fig. 2. The resulting performance plots for all of the models after evaluation.

The above figure (Fig. 2) represents the results achieved at the end of the evaluation process. As it can be seen above, the accuracy performances of the distilled model and both of the GRPO models are extremely different, with the distilled model having an accuracy rating of 44.12%, with the GRPO trained models having accuracy performances of 0.68% (standard training) and 1.48% (custom training). In terms of diversity performance however, they have performed rather similarly with low diversity coefficients, with the distilled model having a diversity coefficient of 0.1123, while the GRPO trained models have diversity coefficients of 0.0839 (standard training) and 0.0888 (custom training).

Additionally, the model that was subjected to the custom GRPO training has been recorded to have a case of reward hacking and policy oscillation, with the training process's mean diversity reward being mostly between 1.50 to 1.00, while having a mean accuracy reward that was mostly below 0.5 with a few spikes upwards 0.5 during several epochs in the middle and near the end of the training session.

IV. Discussion

The evaluation metrics provided in section III presents a severe degradation in accuracy with the GRPO models, with the addition of all models having low diversity coefficients overall.

However, the manual review of the GRPO models' outputs (specifically the custom GRPO trained models') prove that the models are capable of answering the questions accurately enough to disprove the accuracy results (with one example being provided below with Fig. 3). This would, under normal circumstances, mean that the evaluation function is at fault. However, the distilled model's accuracy rating being 44.12% disproves this being the reason, with the reasoning being that the evaluation function being inherently dysfunctional would provide results that were similar to the GRPO models' for the distilled/SFT trained model.

Thus, while the exact reason for this occurrence is unknown, the reasons being either the inherent dysfunctionality of the evaluation function or the inability of the GRPO models can be ruled out. It is hypothesized that the reason may be GRPO models generating invisible tokens in text that break the functionality of the evaluation function, therefore lowering the model's accuracy rating.

```
Expected Target: 14
Actual Output: not include units or words
inside the tag". Let's write it. </think>
Amanda started with 10 notebooks. She
received 6 more, so the total became
$10 + 6 = 16$. She then lost 2 notebooks,
so the final count is $16 - 2 = 14$.
<answer>14</answer>
```

Fig. 3. The example that proves that the GRPO model can reason and solve the mathematical problems provided.

In terms of the diversity coefficients being on the lower end of the scale however, the reason is hypothesized to be the difference in terms of the language of the answers (the dataset's answers are in Turkish, while the teacher model and the student models provided their answers in English upon being prompted to do so). This, in theory, could be solved by making all answers the same language, either via translating the answers in the dataset to be English, or prompting models that are capable of generating answers in Turkish (the models were instructed to answer in English due to their capabilities of mathematical reasoning and providing multilingual output simultaneously being under question). In addition to all this cross-lingual problems, the general solutions of mathematical problems tend to be rather rigid in terms of lexical structure compared to other types of output. Considering the rigidity of the outputs, it isn't unfathomable to theorize that the embedding model considered these outputs to be semantically similar, therefore calculating a higher cosine similarity value for the answers.

V. Conclusion

This project was performed to investigate the effects of rewarding diverse answers that differ from the teacher model's during GRPO training sessions. While the distilled model established a baseline accuracy of 44.12%, the models that were subjected to GRPO trainings showed a near-total collapse in machine-graded accuracy (scoring below 2% overall), with all the models having diversity coefficients that were on the lower end of the scale. However, manual analysis during training and evaluation sessions revealed that the degradation in accuracy and low diversity coefficients were not from the collapse of reasoning capabilities, instead being from reward hacking and the fragility of evaluation processes such as the one performed, due to the models being capable to answer provided questions accurately and the cross-lingual barriers between Turkish and English in terms of diversity.

The future work for this project will include, but is not limited to:

- **Finding the reason for the low accuracy scores during evaluation tests:** As stated in section IV, the exact reason for the low accuracy scores during evaluation is currently unknown, with several reasons being eliminated via process of elimination. Therefore the main priority will be on finding the exact reason for this phenomenon.
- **Fixing the Cross-Language Evaluation Issues:** Another issue that was explained in section IV was the overall low diversity coefficients of all the models tested, and the reason for this phenomenon being theorized to be the language barriers between Turkish and English. Future work in this department will be to fix this issue, either via translating all of the strings to the same language (Turkish or English), or by finding a solution that can handle cross-lingual semantic similarity calculation (one of the potential solutions on this front being theorized is the use of LLM's capable of translation).
- **Improving both the Accuracy and Diversity ratings of the models without the previous issues:** Previous projects performed with the models used in this project have accuracy ratings that exceed the accuracy ratings achieved in this project (the same base model trained via SFT has been recorded to achieve around 60% to 70% accuracy scores in other projects). After the correction of the issues mentioned prior to this point, focus will be on increasing the accuracy and diversity of the models via different training parameters and methods.

Ultimately, this research project shows that while models can learn and perform accurately with diverse answers, evaluation pipelines must evolve past rigid regex extractions lest we end up unable to ascertain the genuine performance of these models.

Acknowledgement

LLM assistance (Gemini 3.1 Pro) was used during the development stage of this project, especially during the bug-fixing process of the project environment and libraries along with performing tweaks to code in small areas (the LLM use is highlighted inside the code provided via GitHub [9]).

Additionally, I would like to thank Yıldız Technical University's Cosmos Research Group for creating the gsm8k_tr dataset, OpenRouter for providing the inference provider service used, OpenAI for creating the teacher model (gpt-oss-120b model), along with the Qwen Team at Alibaba Group for creating the Qwen reasoning (Qwen3.5-4B and Qwen3.6-27B) and embedding (Qwen3-Embedding-0.6B) models that were used in this project in multiple roles.

References

[1] https://www.geeksforgeeks.org/artificial-intelligence/supervised-fine-tuning-sft-for-llms/
[2] https://www.datacamp.com/blog/what-is-grpo-group-relative-policy-optimization
[3] https://www.digitalocean.com/community/conceptual-articles/group-relative-policy-optimization-reinforcement-learning
[4] https://huggingface.co/Qwen/Qwen3.5-4B
[5] https://huggingface.co/datasets/ytu-ce-cosmos/gsm8k_tr
[6] https://huggingface.co/openai/gpt-oss-120b
[7] https://huggingface.co/Qwen/Qwen3.6-27B
[8] https://huggingface.co/Qwen/Qwen3-Embedding-4B
[9] https://github.com/GkslOkn/YTU_CE_Computational_Semantics_Final_Project